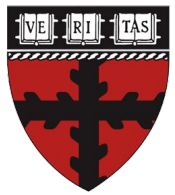
Lecture 4: Containers III

AC215/E115

Pavlos Protopapas
SEAS/Harvard



Logistics / Reminders

1. Please fill this survey.
   https://bit.ly/ac215_2026_survey

   Canvas has survey results



2. When you form a team, please fill this form too.
   https://bit.ly/ac215_2026_groups



Outline

1. Build Optimization
2. Docker Compose
3. Networking
4. Team Workflows

Outline

1. **Build Optimization**
2. Docker Compose
3. Networking
4. Team Workflows

Build Optimization: Why Bother?

Every deploy is a download:

Push to the registry, pull on each server before it can start.

More packages = more to secure & patch:

Build tools and secrets shouldn't ride along.

Slow builds tax the whole team:

Every code change: rebuild → push → pull.

Storage & bandwidth aren't free:

One fat image × every version × every server.

Build Optimization: Why Bother?

Every deploy is a download:

Push to the registry, pull on each server before it can start.

Mor

E

Slo

E

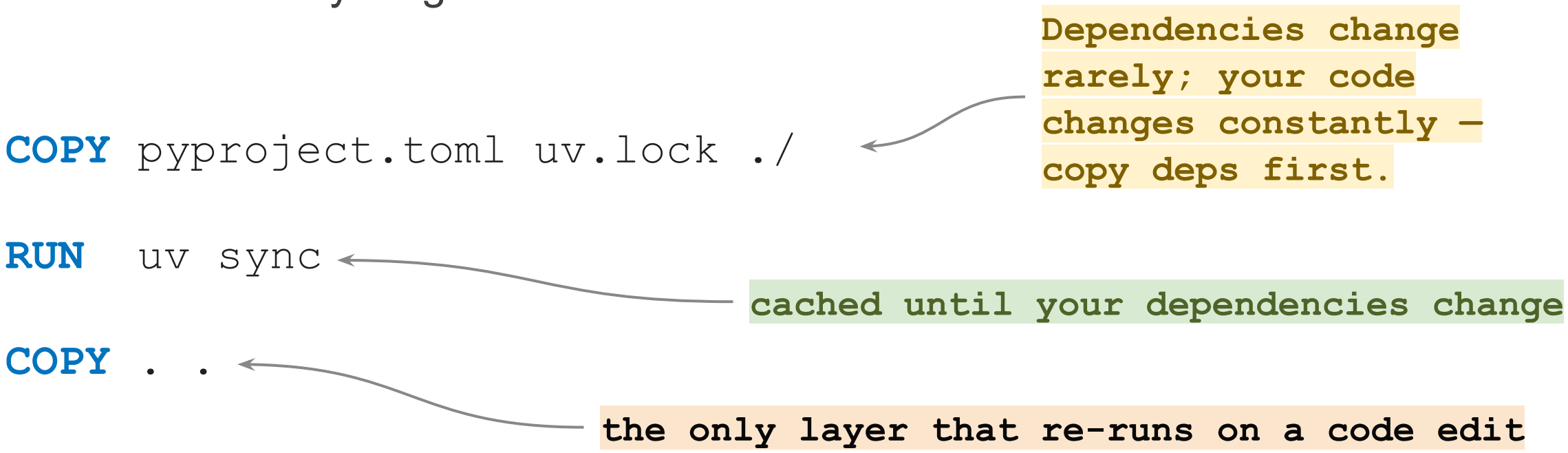
Storage & bandwidth aren't free:

One fat image × every version × every server.

Ship only what you need to RUN, and keep what changes small.

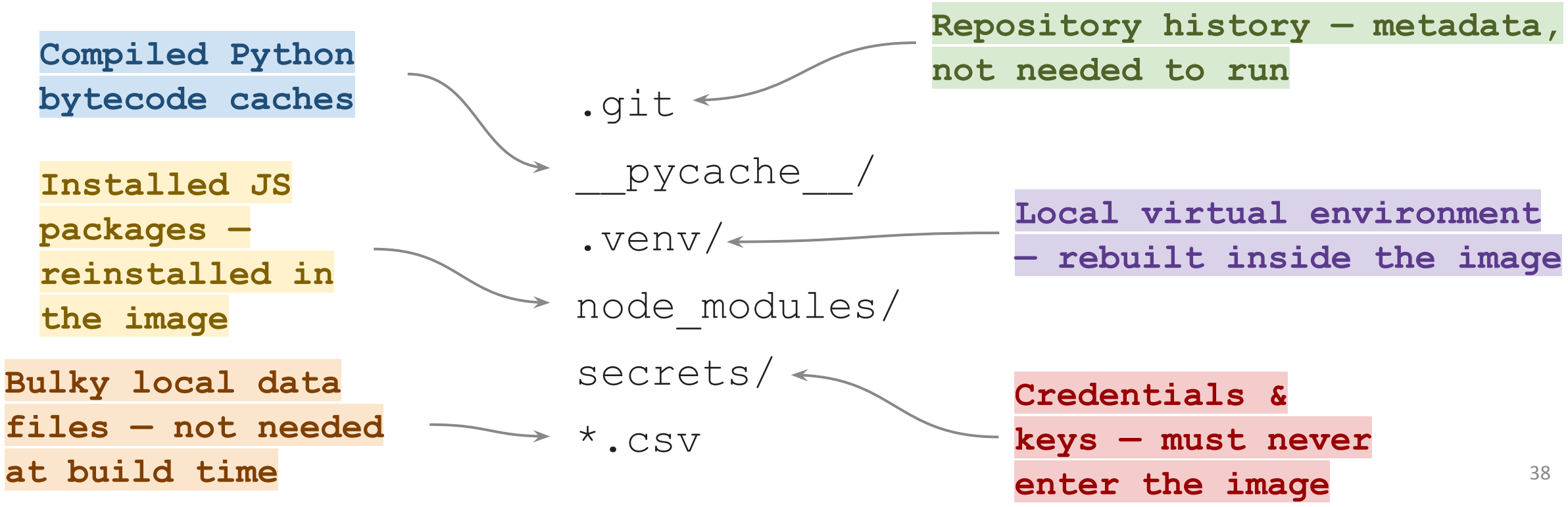
Layer Caching: Order Matters

- Each Dockerfile instruction is a cached layer. Change one layer and you bust the cache for every layer after it.
- So put rarely-changing steps first, frequently-changing steps last.
- Classic mistake: `COPY . .` before installing deps → every code edit reinstalls everything.



`.dockerignore`: Shrink the Build Context

- Like `.gitignore`, but for the **build context** Docker sends to the daemon.
- Each line is a pattern — a file or directory to exclude from the build context.
- Smaller context = faster builds, and you cannot accidentally bake secrets or junk into the image.



How Python Packages Are Installed

- **Ready-to-install package** (wheel): Already built. Pip unpacks and installs it—no compiler needed.
- **Source package:** If no compatible wheel is available, pip builds a wheel from the package's source code, then installs it. Some packages need a compiler and other build tools for this step.

Source distribution

```
cheese_utils-1.0.tar.gz

pyproject.toml
src/
  cheese_utils/
    __init__.py
    prices.py
tests/
  test_prices.py
```



Built wheel

```
cheese_utils-1.0-py3-none-any.whl

cheese_utils/
  __init__.py
  prices.py
cheese_utils-1.0.dist-info/
  METADATA
  WHEEL
  RECORD
```

Build: prepare files for installation.

In this pure-Python example, the wheel contains the same Python code, repackaged with installation metadata—no compiler needed. Packages with native extensions may also require compilation when building their wheels.

Pick a Smaller Base Image

How a dependency installs — and why the base image matters:

- **Prebuilt wheel** (`.whl`) — already built; pip just unpacks it. No compiler needed. Most packages, incl. numpy & pandas.
- **Source** (sdist) — source code, e.g., Python, C, or Rust. If no wheel matches your platform, pip builds one from source. C/Rust extensions require a build toolchain.

Base image	Approx. size	When to use it
`python:3.14`	**~1.63 GB**	Rarely — its full toolchain can compile from source
`python:3.14-slim`	**~217 MB**	**Default — enough when your deps have wheels**
`python:3.14-alpine`	**~86 MB**	Smallest, but musl → often no wheel, so it compiles from source

Slim for most Python; alpine only when you know what you're doing. Smaller base → fewer known vulnerabilities (CVEs).

Pick a Smaller Base Image

How a dependency installs — and why the base image matters:

- With compatible wheels, you can use a smaller image without compilers.
- If a dependency needs compilation, your image must provide the required build tools.
- Wheel compatibility depends on the environment. Alpine may need different wheels from the full and slim images.

Base image	Approx. size	When to use it
`python:3.14`	**~1.63 GB**	Build tools included: gcc, g++, make
`python:3.14-slim`	**~217 MB**	**Good default with compatible wheels**
`python:3.14-alpine`	**~86 MB**	Smaller base. Check that your dependencies support Alpine

Multi-stage builds: Use build tools in the build stage. Leave them out of the final image.

Measure It

Optimize what is actually big — do not guess.

`docker images` — the total size of each image.

```
$ docker images python
REPOSITORY  TAG          SIZE
python      3.14         1.63GB
python      3.14-slim    217MB
```

same Python, a leaner base.

`docker history <image>` — per-layer size; find the fat layer.

```
$ docker history python:3.14-slim
CREATED BY                          SIZE
CMD ["python3"]                       0B
RUN … command symlinks             16.4kB
RUN … build Python + pip           45.2MB
ENV PYTHON_VERSION=3.14.7             0B
RUN … apt-get; runtime deps        13.1MB
ENV PATH, PYTHON_SHA256               0B
# debian.sh … (base image)         109MB
```

Multi-Stage Builds

```
FROM python:3.14 AS builder
WORKDIR /app
RUN pip install uv
COPY pyproject.toml uv.lock ./
RUN  uv sync

FROM python:3.14-slim AS runtime
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY . .
CMD ["/app/.venv/bin/python", "app.py"]
```

Builder stage: Includes common build tools. Uses uv to install dependencies into .venv.

Runtime stage: Starts from slim. Contains only what the app needs to run.

COPY --from=builder: Copies the prepared .venv. Build tools stay in the builder stage.

Builder installs dependencies; copy them into the slim image without the build tools.

Fewer, Leaner Layers

Layers are additive: files deleted in a later layer still take up space in the earlier one.

Trim the extras: --no-install-recommends for apt.

Build-essential: common C/C++ build tools for dependencies built from source.

```
FROM python:3.14-slim
RUN apt-get update \
 && apt-get install -y --no-install-recommends
build-essential \
 && rm -rf /var/lib/apt/lists/*
```

Chain related commands and clean up in the same layer with &&.

A separate cleanup layer does not shrink the image – Delete the downloaded package lists in the same RUN step to save space. The installed build tools remain available.

Building for the Target Platform

Docker build already uses Buildx and BuildKit. Choose the platform where your app will run.

--platform builds for the target's CPU arch, not your laptop's.

```
$ docker build --platform linux/amd64 -t myapp .
```

```
$ docker build --push \
      --platform linux/amd64,linux/arm64 -t myapp .
```

--push: Push the multi-platform image to a registry.

Apple Silicon = arm64
Intel / AMD = amd64
Clouds = arm64 or amd64
Check the deployment target.

Build Secrets: Tokens for Dockerfile RUN Steps

When a Dockerfile RUN step needs a token, supply it through a temporary secret mount so the token itself stays out of the Dockerfile and image.

```
# syntax=docker/dockerfile:1
RUN --mount=type=secret,id=gh_token \
    GH_TOKEN=$(cat /run/secrets/gh_token) &&\

    pip install "git+https://GH_USERNAME:${GH_TOKEN}@github.com/org/lib"
```

Enables --mount — must be the Dockerfile's first line.

Names the secret — reuse this id on docker build.

A container path — mounted here only for this build step.

$(…) runs cat so it reads that file into GH_TOKEN.

Build Secrets: Supply the Token from Your Terminal

Supply the token file when starting the build, using the same secret ID as the Dockerfile RUN step.

A local path — the token file on your machine.

Same id as the RUN --mount — that's the link.

```
$ docker build --secret id=gh_token,src=$HOME/.gh_token .
```

Avoid these ways of passing secrets:

Avoid these	Risk for secrets
`ENV GH_TOKEN=ghp_xxx`	Stored in image configuration
`ARG GH_TOKEN`	May appear in history or build metadata
`COPY .gh_token /tmp/token`	Remains in earlier layer after later deletion

Outline

1. Build Optimization
2. **Docker Compose**
3. Networking
4. Team Workflows

The Problem Compose Solves

A real app is many containers — starting them by hand is a chore:

- One `docker run` per service — each with its own `-p`, `-v`, `-e`, `--network` flags.
- You create the network and start the services in the right order, yourself.
- Every code change: stop → {remove} → rebuild → run, service by service.

Our example — a two-container UPPER-case transformer:




```
docker network create app-net
docker build -t processor-img ./processor
docker run -d --network app-net --name processor processor-img
docker build -t gateway-img ./gateway
docker run -d --network app-net -p 8090:9100 \
  -e PROCESSOR_HOST=processor -e PROCESSOR_PORT=9200 \
  -v "$PWD/gateway:/app" --name gateway gateway-img
```

The Problem Compose Solves

A real app is many containers — starting them by hand is a chore:

- One `docker run` per service — each with its own `-p`, `-v`, `-e`, `--network` flags.
- You create the network and start the services in the right order, yourself.
- Every code change: stop → {remove} → rebuild → run, service by service.

One multi-container app deserves one file and one command.

What Is Docker Compose?

One tool + one file to define and run a multi-container app.

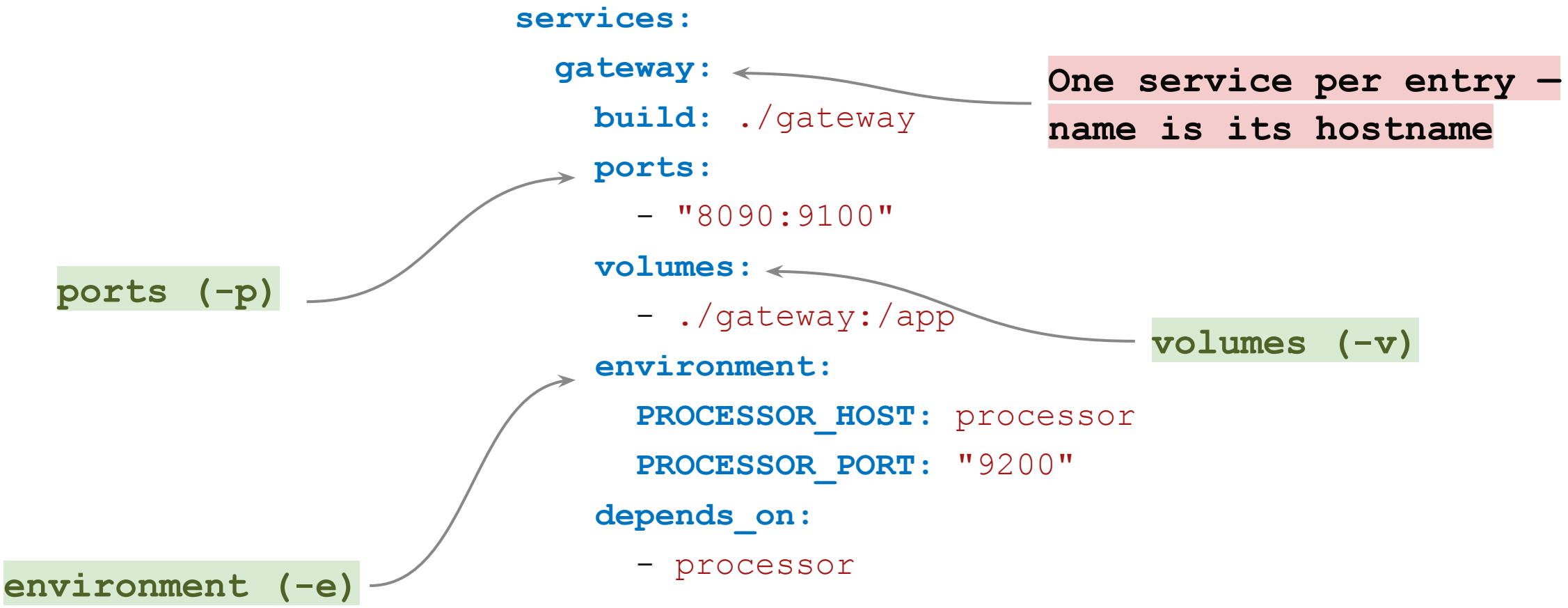
- **One file** — `compose.yaml` describes every service, network, and volume (legacy name: `docker-compose.yaml`).
- **One tool** — `docker compose`, built into Docker.

Describe services in `compose.yaml`; docker compose up starts them, building if needed.

Task	By hand	With Compose
Network	`docker network create + attach each`	**automatic, shared**
Start it all	`N x docker run (in order)`	**`docker compose up`**
Tear down	`stop + rm each`	**`docker compose down`**

compose.yaml

From separate Docker commands to one Compose file.



build: vs image:

`build` vs `image` — each service picks where its image comes from.

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      - processor

  processor:
    image: ac215/processor:1.0
```

Compose builds an image from ./gateway using its Dockerfile and local files.

Pull a prebuilt image from a registry, such as Docker Hub.

Networking for Free

Your gateway calls the processor by its name — you just ran it:



You never ran this:

```
$ docker network create app-net
$ docker run --network app-net --name gateway …
```

Compose creates the network for you.

Volumes

Attach storage to a service — just a list under the `volumes:` key.

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./secrets:/secrets:ro
      - models:/models
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      - processor

volumes:
  models:
```

Bind mount — host path in

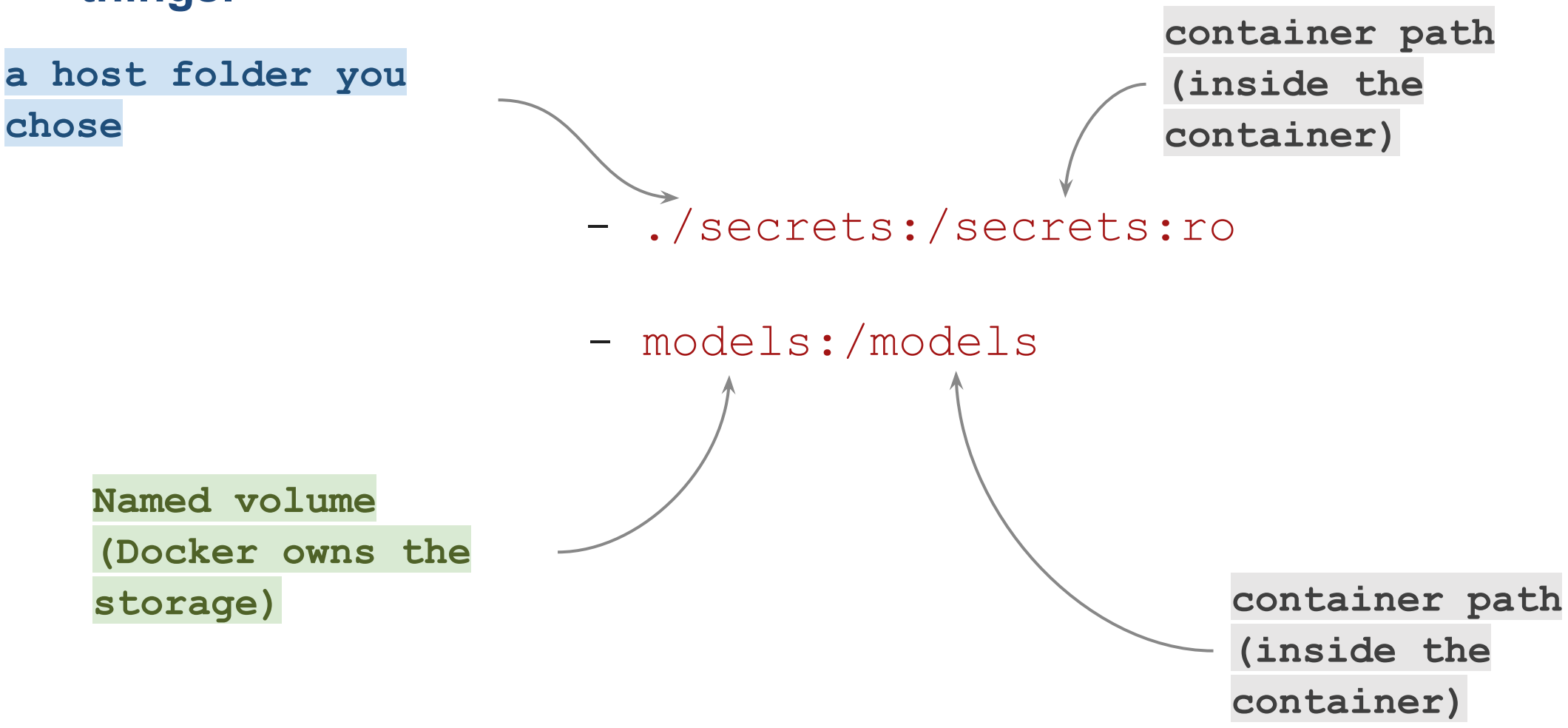
:ro = read-only

Named volume — Docker persists it, just a name

Declare the named volume here

Volumes: Bind Mount vs Named Volume

Same `source:container` shape — but the left side means two different things.



Environment

Two ways to pass variables into a service — both injected at run time.

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    env_file:
      - .env
```

Set vars inline, like -e (a few non-secret values)

Load a file (.env) kept out of git; secrets stay out of the YAML

Environment: Build Time vs Run Time

Where you write a variable decides when it exists.

Where you write it	Compose file (${VAR})	Build	Image	Container (run)
`Dockerfile: ENV`	—	✓	✓	✓
`Dockerfile: ARG`	—	✓	—	—
`Compose: build.args`	—	✓	—	—
`Compose: environment: / env_file:`	—	—	—	✓
`.env file (auto-loaded)`	✓	—	—	—
`Shell variable $VAR`	✓	—	—	—

```
# .env
TAG=1.0

# Or in the shell
export TAG=1.0
```

fills →

```
#docker-compose.yml
services:
  processor:
    image: ac215/processor:${TAG}
```

Startup Order

You've seen `gateway` depend on `processor` — what does it guarantee?

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      - processor

  processor:
    image: ac215/processor:1.0
```

Processor comes "up" first, then gateway.

But "up" = container started, not app ready.

Startup Order

Real readiness: wait until processor is healthy, not just started.

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      processor:
        condition: service_healthy

  processor:
    image: ac215/processor:1.0
    healthcheck:
      test: curl -f localhost:9200/health
```

A fixed Compose value, not a name you invent.
The other two:
service_started
service_completed_successfully

Order — processor comes "up" first, then gateway.

hold gateway until the check passes.

healthcheck — the probe that decides when processor is ready.

Everyday Compose Commands

Run these from the directory holding your `docker-compose.yml`:

```
docker compose up
docker compose up -d
docker compose up --build
docker compose ps -a
docker compose logs -f
docker compose down
docker compose down -v
```

`up` — build images, create the network, and start every service.

`-d` — run detached, in the background.

`--build` — rebuild the image first.

`ps -a` — which services are up, and which have stopped.

`logs -f` — watch every service's output live, in one stream.

`down` — stop and remove all containers and the network at once.

`-v` — also delete named volumes (your data). Delete!

The Whole App in One File

```yaml
services:
  gateway:
    build: ./gateway
    ports:
      - "8090:9100"
    volumes:
      - ./gateway:/app
    environment:
      PROCESSOR_HOST: processor
      PROCESSOR_PORT: "9200"
    depends_on:
      - processor

  processor:
    image: ac215/processor:1.0
```

The whole app — `gateway` + `processor` — in one file. `docker compose up` runs all of it.

When Compose Isn't Enough

- Compose runs on **one machine** — great for dev and a single small server.
- No multi-machine scaling, self-healing, rolling updates, or cross-node load balancing.
- Production, multi-host → **Kubernetes** (later in the course) — same “declare services, find by name” idea.

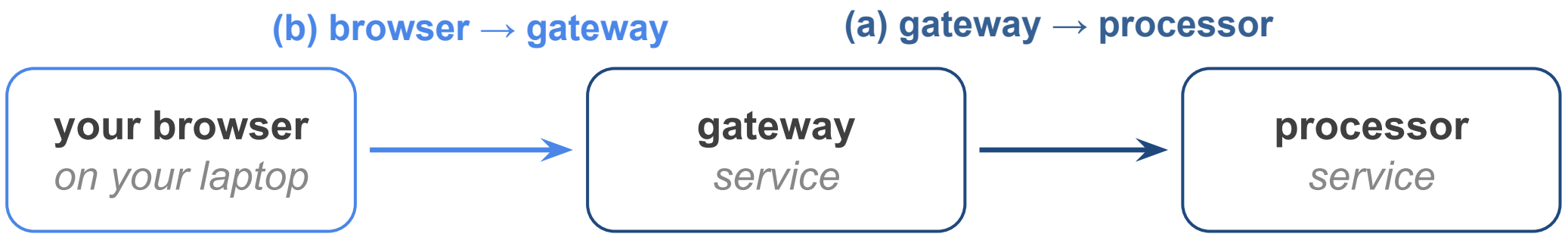
Aspect	Docker Compose	Kubernetes
Scope	one machine	many machines (a cluster)
Great for	dev + a single small server	production scale, self-healing, rolling updates

Outline

1. Build Optimization
2. Docker Compose
3. **Networking**
4. Team Workflows

Your Containers Need to Talk

Compose starts many containers — but two questions decide whether anything works:

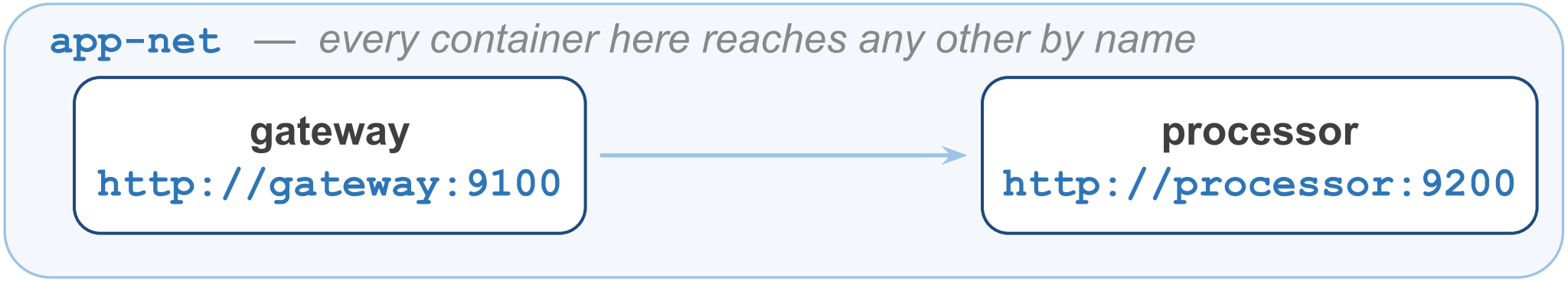


Arrows show request direction; responses return over the same connections.

- **(a) Gateway → processor** — one service needs to reach another.
- **(b) Browser → gateway** — you, at your browser, need to reach the app from outside.
- **Isolated by default** — each container has its own network namespace (Containers I) until you wire these paths.

Gateway → Processor: One Shared Network

Put them on one network → they reach each other by name.




```
1 — create one shared network
$ docker network create app-net

2 — attach each container & name it
$ docker run --network app-net \
      --name gateway    gateway-img
$ docker run --network app-net \
      --name processor  processor-img

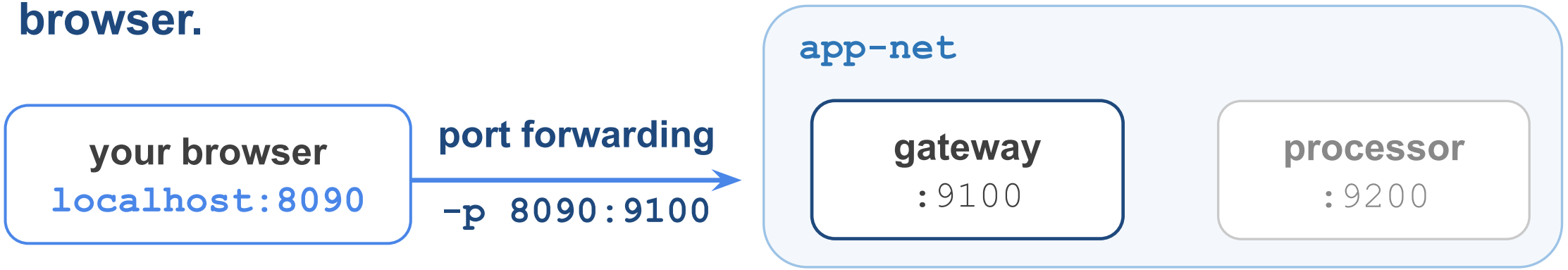
3 — reach it by name — Docker DNS, no IP
POST http://processor:9200/
```

Optional here: no container calls gateway by name.

Compose does all 3 for free — It creates the network, connects the containers, and makes services reachable by name.

Browser → Gateway: Publish a Port

Publish a port to reach a container from your laptop — open its page in a browser.



8090 : 9100 = HOST : CONTAINER

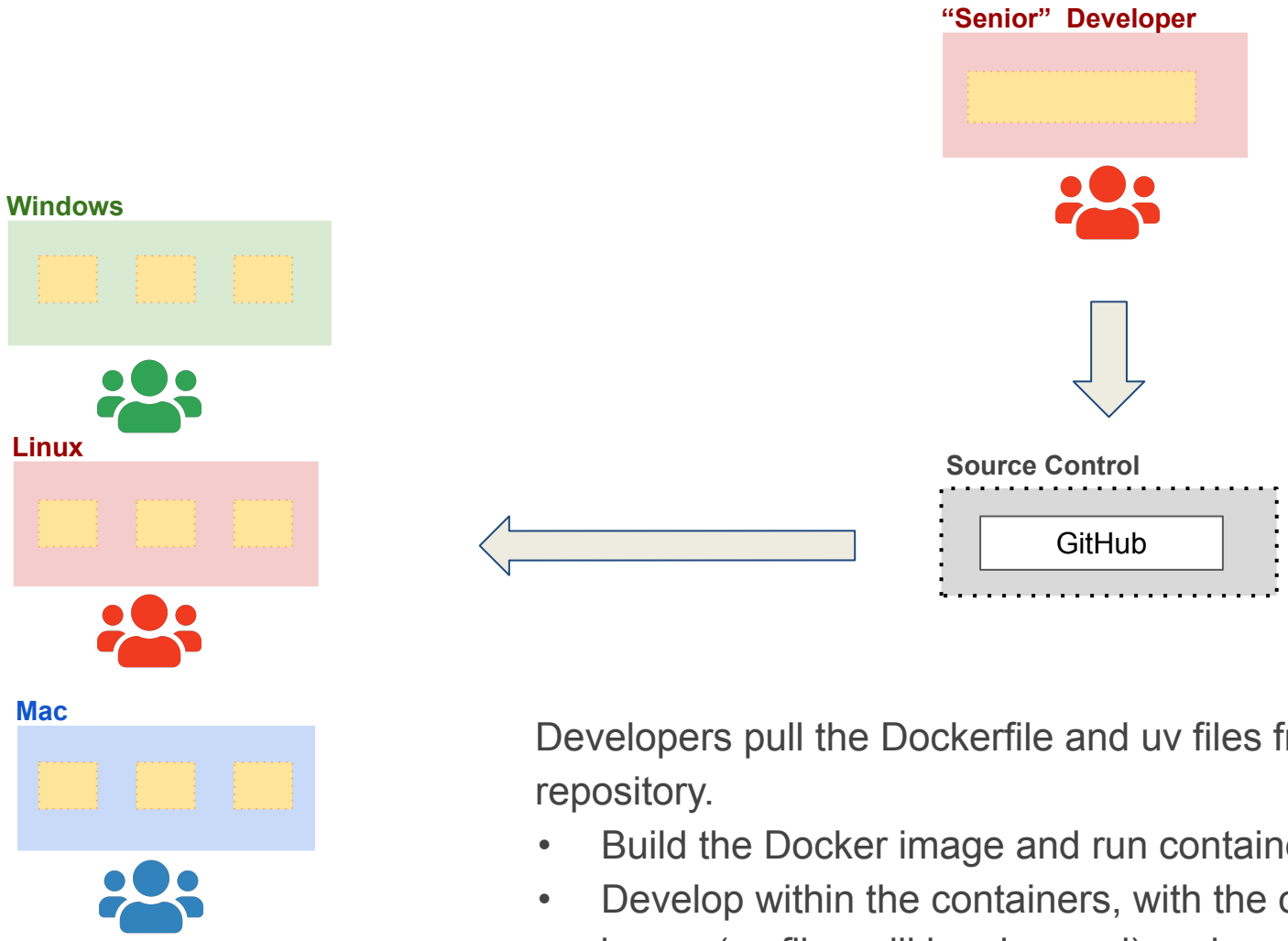
```
# docker run command
$ docker run --network app-net \
        --name gateway \
    -p 127.0.0.1:8090:9100 gateway-img
```

```
# Compose ports excerpt
    ports:
      - "127.0.0.1:8090:9100"
```

Outline

1. Build Optimization
2. Docker Compose
3. Networking
4. **Team Workflows**

Workflow with Docker: Scenario 1 (early stages of development)



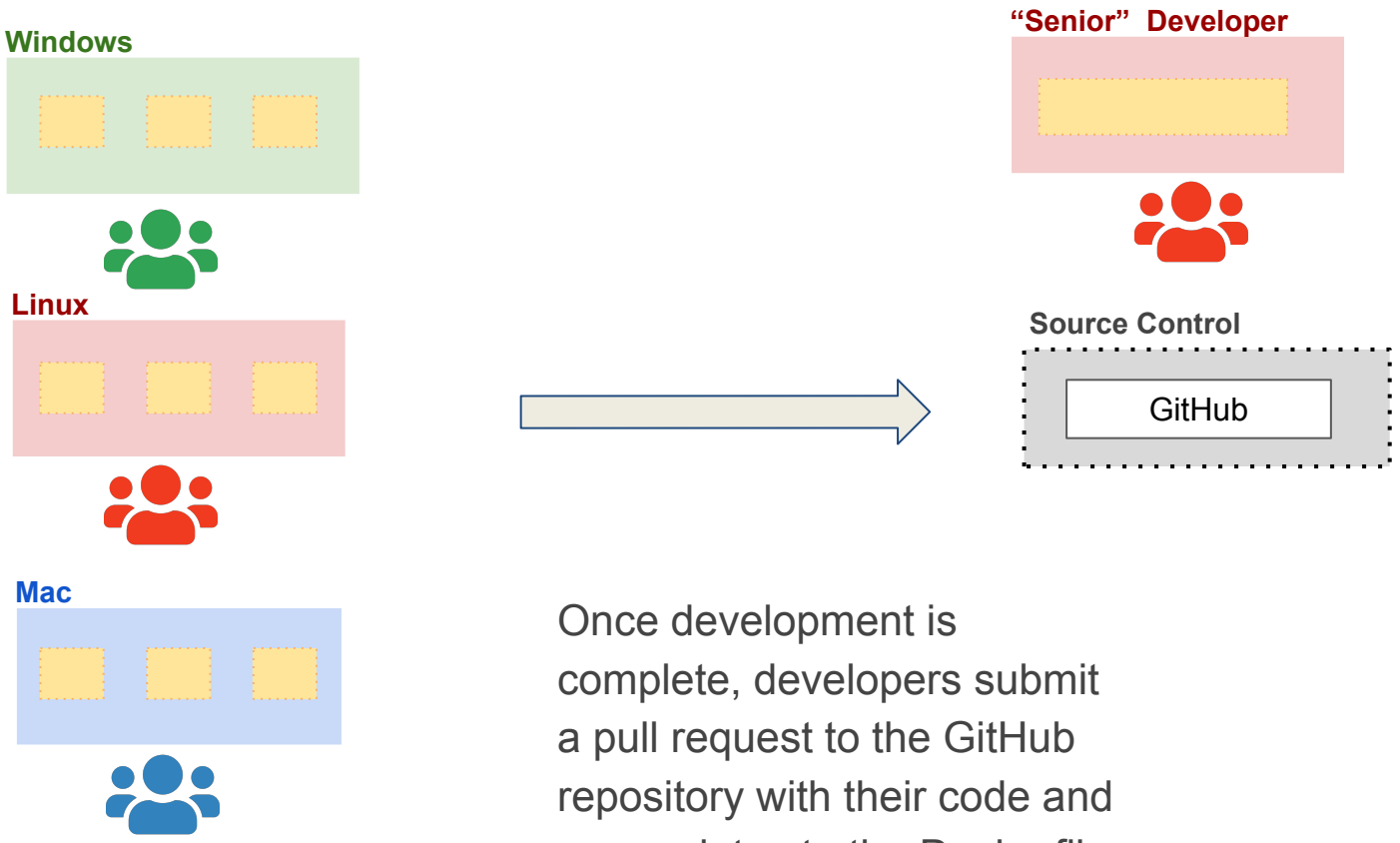
A senior developer is responsible for creating the Dockerfile, along with the initial pyproject.toml and uv.lock.

Push Dockerfile and uv files to the GitHub repository

Developers pull the Dockerfile and uv files from the GitHub repository.

- Build the Docker image and run containers.
- Develop within the containers, with the option to install packages (uv files will be changed) and modify the Dockerfile if needed, then rebuild.

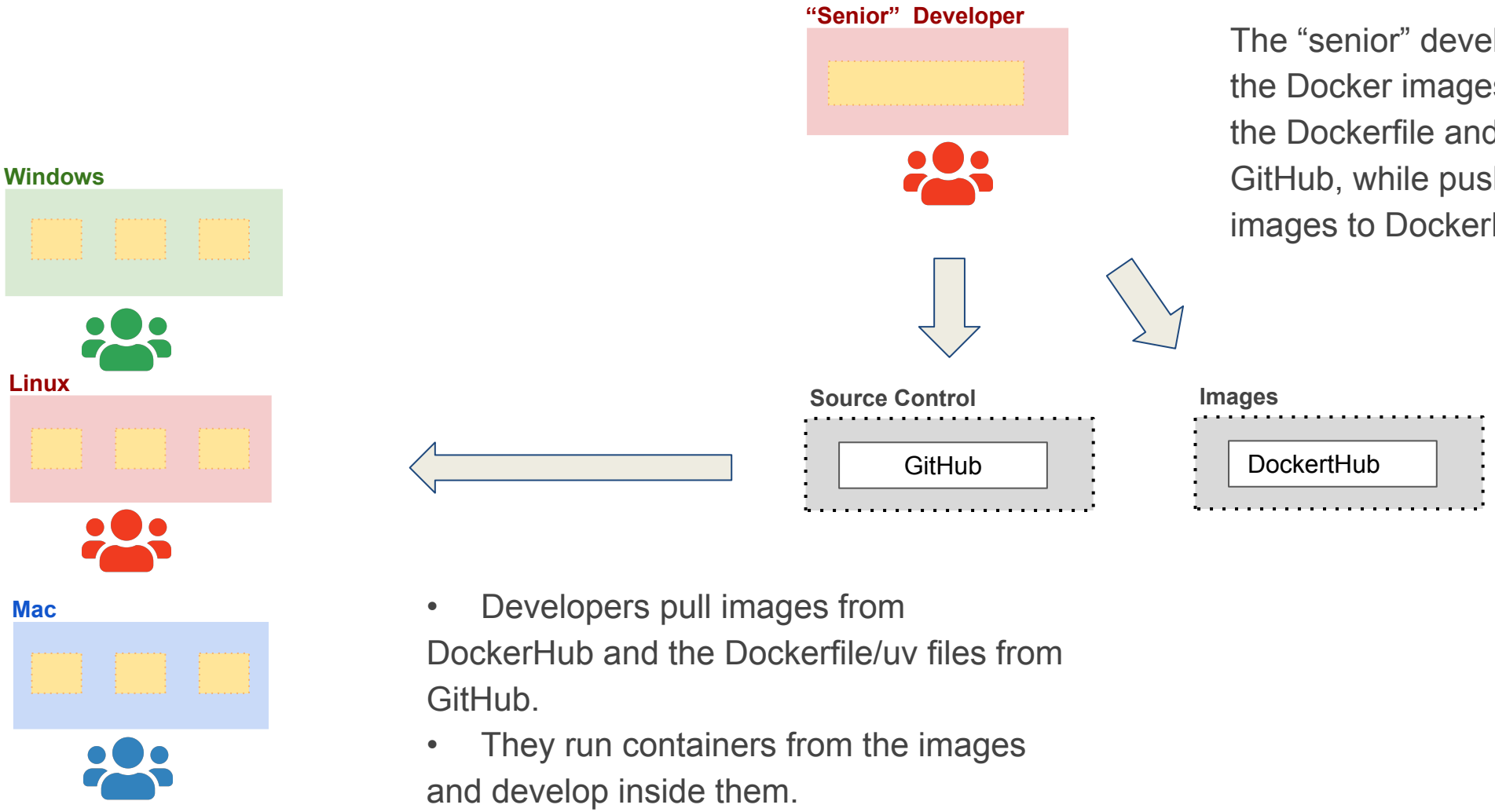
Workflow with Docker: Scenario 1 (early stages of development)



Once development is complete, developers submit a pull request to the GitHub repository with their code and any updates to the Dockerfile or uv files.

The senior developer reviews and merges all pull requests, then consolidates the updates into a new Dockerfile and uv files.

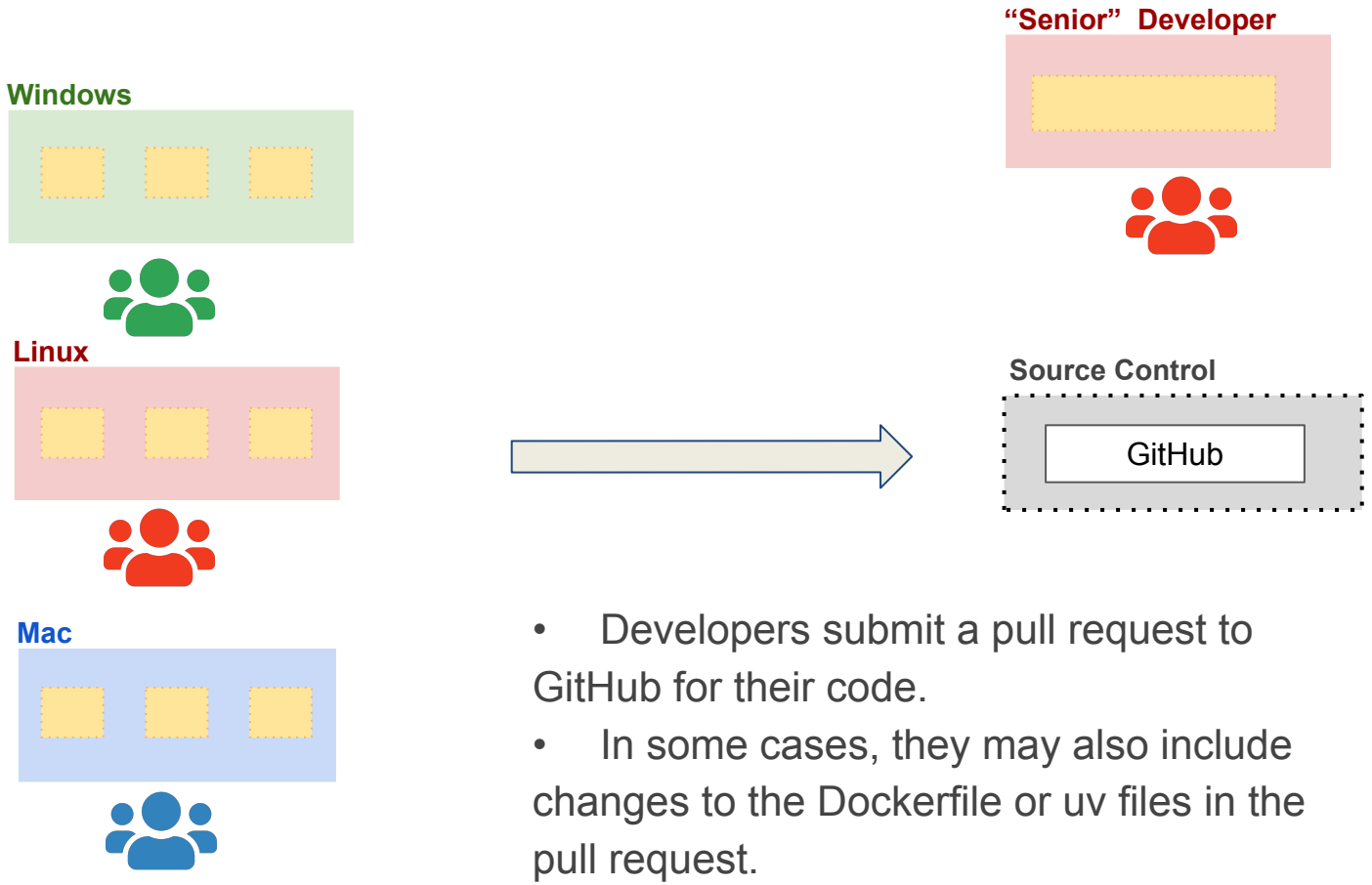
Workflow with Docker: Scenario 2 (later stages of development)



The "senior" developer creates the Docker images and pushes the Dockerfile and uv files to GitHub, while pushing the images to DockerHub.

- Developers pull images from DockerHub and the Dockerfile/uv files from GitHub.
- They run containers from the images and develop inside them.
- Rebuilding the images is only needed on rare occasions.

Workflow with Docker: Scenario 2 (later stages of development)



The senior developer reviews and merges all pull requests, then consolidates the updates into a new Dockerfile and uv files and builds new images which are pushed to DockerHub.

- Developers submit a pull request to GitHub for their code.
- In some cases, they may also include changes to the Dockerfile or uv files in the pull request.

Workflow with Docker: A Flexible Approach

Is there a “perfect” workflow?

No

So, how do we decide what to do?

Clear communication and rules are essential. Each team can have its own workflow, based on the project and team needs.

Tutorial: **Mega-pipeline Flexible Workflow**

Repeat the Mega Pipeline workflow—this time using your own Google Cloud resources.

You set up:

- Your own Cloud Storage bucket
- Your secrets and authentication
- The required Google Cloud APIs

We provide:

- `Dockerfile`, `pyproject.toml`, and `uv.lock`
- A shell script to build and run the container for development or production

Once your cloud setup is complete, build and run using the provided script.

Scan to Access Repository



GitHub Link:

https://github.com/dlops-io/mega-pipeline/tree/flexible-workflow

Tutorial: **PORT TOY**

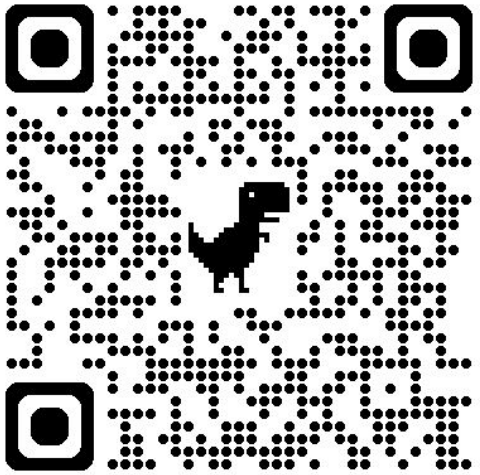
Container Networking & Docker Compose

In this tutorial, we explore:

- How containers communicate on an internal network
- How Docker Desktop forwards localhost traffic to containers
- How host ports differ from container ports
- How Docker Compose sets up and runs it all

How about this shorter version? In this tutorial, we explore container networking and Docker Compose: internal traffic, localhost forwarding, host versus container ports, and how Compose runs the setup.

Scan to Access Repository



https://github.com/dlops-io/port_toy#-running-the-app



THANK YOU